

DEV_IMPL_GUIDE - SUB-WF-PAUSE-01-FLOW-WIK

1. Purpose

This guide explains how to implement and test the WIK pause BPMN as the flow executes at runtime.

Use it with:

- DD_SUB-WF-PAUSE-01-FLOW-WIK-v1.0.md
- DD_SUB-WF-PAUSE-01-Subscription_Pause-v1.0.md
- DD_SUB-WF-FRAMEWORK-01-Subscription_Workflow_Framework-v1.0.md
- DD_SUB-WF-FRAMEWORK-01-WORKERS-v1.0.md

This guide is specific to the BPMN process file:

sub-pause-wik.bpmn

2. Runtime Actors

Runtime component	Role in this flow
subscription-workflow-api pod	Receives WIK pause request and calls the shared operation framework.
SUB-LM service/module	Provides authoritative subscription summary and status mutation APIs.
Pause workflow persistence	Owns pause reason/config/history reads and writes, unless implemented inside the same module as SUB-LM.
Drools	Evaluates rules.subscription.pause.wik.
Camunda	Runs process sub-pause-wik.
Worker apps	Execute external task topics from the BPMN. One app can handle many topics.
BIL-01	Evaluates PAUSE_INTENT and optional pause fee.
FUL-04	Suspends service.
Message correlators	Correlate payment and fulfillment messages back to Camunda.
NOT-01	Sends customer notification after success.

3. Before The Flow Can Run

Confirm these objects exist before deploying or testing the BPMN:

Object	Required value
BPMN process key	sub-pause-wik
Operation config	(operator_code=WIK, operation_kind=PAUSE) -> sub-pause-wik
Pause config	WIK row in subscription_pause_config
Reason catalog	WIK reasons in pause_reason
Drools package	rules.subscription.pause.wik
Worker subscriptions	Required worker topics from the WIK DD
Message correlators	payment-confirmed, fulfillment-completed, fulfillment-failed

4. Step 1 - Caller Sends Request

Public endpoint:

```
POST /api/subscriptions/sub_01HZ_KAMAU_FIBER/pause HTTP/1.1
Authorization: Bearer <jwt-or-service-token>
Idempotency-Key: pause-sub_01HZ_KAMAU-vacation-20260315
Content-Type: application/json
```

```
{
  "operatorCode": "WIK",
  "pauseTrigger": "CUSTOMER_REQUESTED_PAUSE",
  "pauseReasonCode": "CUSTOMER_VACATION",
  "durationDays": 31,
```

```

    "notes": "Customer travelling to Mombasa for holiday season",
    "initiatingWorkflowKind": "BO_UI",
    "initiatingWorkflowRef": "cc-ticket-2026-03-15-3301"
  }

```

The request body does not decide the actor role. Actor identity and role must come from the authenticated user or service token.

5. Step 2 - API Pod Validates Shape And Idempotency

The subscription-workflow-api pod validates:

Field	Validation
operatorCode	Required and must be WIK for this BPMN.
pauseTrigger	CUSTOMER_REQUESTED_PAUSE or ADMIN_PAUSE.
pauseReasonCode	Required.
openEnded, durationDays, resumeScheduledAt	At most one mode selector supplied.
durationDays	Positive integer when supplied.
resumeScheduledAt	Future timestamp when supplied.

Idempotency behavior:

- First request inserts the operation and starts BPMN.
- Same Idempotency-Key with same request hash returns the existing operation response.
- Same Idempotency-Key with a different request hash must return 409 IDEMPOTENCY_KEY_REUSED.

Implementation note:

Do not insert a new operation before checking existing key and request hash. Use a transaction or unique constraint on the idempotency key plus operator/operation scope so concurrent duplicate requests cannot create two operations.

6. Step 3 - API Loads Required Data

6.1 Subscription Summary

The API gets current subscription state from SUB-LM.

Same deployable:

```
SubscriptionReadService.getSummary(subscriptionId)
```

Split services:

```

GET /internal/subscriptions/sub_01HZ_KAMAU_FIBER/summary
Authorization: Bearer <subscription-workflow-service-token>

```

Expected response:

```

{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "operatorCode": "WIK",
  "customerId": "cust_01HZ_KAMAU",
  "statusCode": "ACTIVE",
  "packageRef": "pkg_FIBER_HOME_50M",
  "homepassId": "hp_01HZ_KAREN_017",
  "activeRestrictions": []
}

```

Do not use cache for current subscription status. Pause is a state-changing workflow and must use the authoritative SUB-LM read path.

6.2 Pause Config

Read from pause workflow persistence:

```

SELECT *
FROM subscription_pause_config
WHERE operator_code = 'WIK';

```

Required sample:

```
{
```

```

    "operatorCode": "WIK",
    "customerSelfServiceEnabled": true,
    "scheduledPauseEnabled": true,
    "maxFutureScheduledResumeDays": 90,
    "minPauseHours": 24,
    "customerNotificationEnabled": true
}

```

6.3 Pause Reason

Read the requested reason:

```

SELECT *
FROM pause_reason
WHERE code = 'CUSTOMER_VACATION'
AND is_active = TRUE;

```

Required sample:

```

{
  "code": "CUSTOMER_VACATION",
  "customerSelfServiceEligible": true,
  "adminOnly": false,
  "maxDurationDays": 90,
  "openEndedAllowed": false,
  "pausesBillCycle": true,
  "suspendsFulfillment": true,
  "defaultFeeBillableEventCode": null
}

```

6.4 Open Pause Check

Check that the subscription does not already have an open pause:

```

SELECT pause_id, pause_reason_code, suspended_at, resume_scheduled_at
FROM subscription_pause_history
WHERE subscription_id = :subscriptionId
AND actual_resume_at IS NULL;

```

If a row exists, return:

```

409 Conflict

{
  "errorCode": "ALREADY_PAUSED",
  "message": "Subscription already has an open pause."
}

```

7. Step 4 - API Evaluates Drools Before Starting BPMN

Call Drools package:

```
rules.subscription.pause.wik
```

Facts:

```

{
  "operatorCode": "WIK",
  "subscription": {
    "subscriptionId": "sub_01HZ_KAMAU_FIBER",
    "statusCode": "ACTIVE"
  },
  "request": {
    "pauseTrigger": "CUSTOMER_REQUESTED_PAUSE",
    "pauseReasonCode": "CUSTOMER_VACATION",
    "durationDays": 31,
    "openEnded": false,
    "resumeScheduledAt": null
  },
  "reason": {
    "code": "CUSTOMER_VACATION",
    "customerSelfServiceEligible": true,
    "adminOnly": false,
    "maxDurationDays": 90,
    "openEndedAllowed": false,
    "pausesBillCycle": true,
    "suspendsFulfillment": true,
    "defaultFeeBillableEventCode": null
  },
  "config": {
    "customerSelfServiceEnabled": true,
    "scheduledPauseEnabled": true,

```

```

        "maxFutureScheduledResumeDays": 90,
        "minPauseHours": 24
    },
    "actor": {
        "userId": "cc-agent-wik-007",
        "role": "CALL_CENTER"
    },
    "runtime": {
        "now": "2026-03-15T10:00:00Z"
    }
}

```

Expected decision:

```

{
  "eligible": true,
  "durationValid": true,
  "durationMode": "FIXED_DAYS",
  "durationDays": 31,
  "resumeScheduledAt": "2026-04-15T10:00:00Z",
  "pauseFeeApplicable": false,
  "reasonCatalogSnapshot": {
    "code": "CUSTOMER_VACATION",
    "customer_self_service_eligible": true,
    "admin_only": false,
    "max_duration_days": 90,
    "open_ended_allowed": false,
    "pauses_bill_cycle": true,
    "suspends_fulfillment": true,
    "default_fee_billable_event_code": null
  }
}

```

If Drools rejects, return 409 or 422 according to parent API error mapping and do not start Camunda.

8. Drools Rules To Configure

8.1 Active Subscription

```

rule "WIK pause requires ACTIVE subscription"
when
  $req: PauseRequestFact()
  $sub: SubscriptionStateFact(subscriptionId == $req.subscriptionId, statusCode != "ACTIVE")
then
  modify($req) {
    setEligible(false);
    setEligibilityReasonCode("INVALID_STATE_FOR_PAUSE")
  }
end

```

8.2 No Open Pause

```

rule "WIK pause requires no open pause history"
when
  $req: PauseRequestFact(eligible == null)
  exists (PauseHistoryFact(subscriptionId == $req.subscriptionId, actualResumeAt == null))
then
  modify($req) {
    setEligible(false);
    setEligibilityReasonCode("ALREADY_PAUSED")
  }
end

```

8.3 Trigger Role Gating

```

rule "WIK customer requested pause allowed roles"
when
  $req: PauseRequestFact(
    pauseTrigger == "CUSTOMER_REQUESTED_PAUSE",
    eligible == null,
    initiatingActorRole not in ("CUSTOMER_SELF_SERVICE", "CALL_CENTER", "SUBSCRIPTION_OPERATOR")
  )
then
  modify($req) {
    setEligible(false);
    setEligibilityReasonCode("ROLE_NOT_AUTHORIZED_FOR_TRIGGER")
  }
end

rule "WIK admin pause allowed roles"

```

```

when
  $req: PauseRequestFact(
    pauseTrigger == "ADMIN_PAUSE",
    eligible == null,
    initiatingActorRole not in ("SUBSCRIPTION_ADMIN", "SUBSCRIPTION_OPERATOR")
  )
then
  modify($req) {
    setEligible(false);
    setEligibilityReasonCode("ROLE_NOT_AUTHORIZED_FOR_TRIGGER")
  }
end

```

8.4 Reason Eligibility

```

rule "WIK pause reason must exist and be active"
when
  $req: PauseRequestFact(eligible == null)
  not (PauseReasonFact(code == $req.pauseReasonCode, isActive == true))
then
  modify($req) {
    setEligible(false);
    setEligibilityReasonCode("UNKNOWN_OR_INACTIVE_PAUSE_REASON")
  }
end

rule "WIK customer self-service cannot use admin-only reason"
when
  $req: PauseRequestFact(initiatingActorRole == "CUSTOMER_SELF_SERVICE", eligible == null)
  $reason: PauseReasonFact(code == $req.pauseReasonCode, adminOnly == true)
then
  modify($req) {
    setEligible(false);
    setEligibilityReasonCode("ADMIN_ONLY_PAUSE_REASON")
  }
end

rule "WIK customer self-service requires eligible reason"
when
  $req: PauseRequestFact(initiatingActorRole == "CUSTOMER_SELF_SERVICE", eligible == null)
  $reason: PauseReasonFact(code == $req.pauseReasonCode, customerSelfServiceEligible == false, adminOnly == false)
then
  modify($req) {
    setEligible(false);
    setEligibilityReasonCode("PAUSE_REASON_NOT_CUSTOMER_SELF_SERVICE")
  }
end

```

8.5 Duration Resolution

```

rule "WIK fixed days duration within bounds"
when
  $req: PauseRequestFact(durationDays != null, durationValid == null)
  $reason: PauseReasonFact(code == $req.pauseReasonCode)
  $cfg: OperatorConfigFact()
  eval($req.getDurationDays() * 24 >= $cfg.getMinPauseHours()
    && ($reason.getMaxDurationDays() == null || $req.getDurationDays() <= $reason.getMaxDurationDays())
    && $req.getDurationDays() <= $cfg.getMaxFutureScheduledResumeDays())
then
  java.time.Instant resumeAt = java.time.Instant.now().plusSeconds($req.getDurationDays() * 86400L);
  modify($req) {
    setDurationMode("FIXED_DAYS");
    setResumeScheduledAt(resumeAt.toString());
    setDurationValid(true)
  }
end

rule "WIK fixed days duration out of bounds"
when
  $req: PauseRequestFact(durationDays != null, durationValid == null)
  $reason: PauseReasonFact(code == $req.pauseReasonCode)
  $cfg: OperatorConfigFact()
  eval(!($req.getDurationDays() * 24 >= $cfg.getMinPauseHours()
    && ($reason.getMaxDurationDays() == null || $req.getDurationDays() <= $reason.getMaxDurationDays())
    && $req.getDurationDays() <= $cfg.getMaxFutureScheduledResumeDays()))
then
  modify($req) {
    setDurationValid(false);
    setDurationReasonCode("DURATION_OUT_OF_BOUNDS")
  }
end

```

8.6 Fee Applicability

```
rule "WIK pause fee applies when customer reason has default fee"
when
  $req: PauseRequestFact(pauseTrigger == "CUSTOMER_REQUESTED_PAUSE")
  $reason: PauseReasonFact(code == $req.pauseReasonCode, defaultFeeBillableEventCode != null)
then
  modify($req) {
    setPauseFeeApplicable(true)
  }
end

rule "WIK pause fee default false"
salience -100
when
  $req: PauseRequestFact(pauseFeeApplicable == null)
then
  modify($req) {
    setPauseFeeApplicable(false)
  }
end
```

9. Step 5 - API Starts Shared Operation

After validation and Drools approval, the API asks the shared SUB-WF framework to create the operation and start Camunda.

Same deployable:

```
OperationStartService.start(StartOperationCommand command)
```

Split services:

```
POST /internal/subscription-operations/start
Authorization: Bearer <subscription-workflow-service-token>
Content-Type: application/json
```

The payload below is the internal command passed to the framework start contract. It is not the public pause API request.

```
{
  "operatorCode": "WIK",
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "operationKind": "PAUSE",
  "idempotencyKey": "pause-sub_01HZ_KAMAU-vacation-20260315",
  "initiatingActorUserId": "cc-agent-wik-007",
  "initiatingActorRole": "CALL_CENTER",
  "initiatingWorkflowKind": "BO_UI",
  "initiatingWorkflowRef": "cc-ticket-2026-03-15-3301",
  "bpmnVariables": {
    "customerId": "cust_01HZ_KAMAU",
    "pauseTrigger": "CUSTOMER_REQUESTED_PAUSE",
    "pauseReasonCode": "CUSTOMER_VACATION",
    "durationMode": "FIXED_DAYS",
    "durationDays": 31,
    "resumeScheduledAt": "2026-04-15T10:00:00Z",
    "reasonCatalogSnapshot": {
      "code": "CUSTOMER_VACATION",
      "pauses_bill_cycle": true,
      "suspends_fulfillment": true,
      "max_duration_days": 90
    },
    "pauseFeeApplicable": false,
    "notes": "Customer travelling to Mombasa for holiday season"
  }
}
```

The framework:

1. Inserts subscription_operation.
2. Resolves (WIK, PAUSE) to sub-pause-wik.
3. Starts Camunda process sub-pause-wik.
4. Stores the Camunda process instance id against the operation.

API response:

```
{
  "operationId": "subop_01HZ_KAMAU_PAUSE",
  "operationKind": "PAUSE",
  "bpmnProcessKey": "sub-pause-wik",
  "currentState": "INITIATED",
}
```

```

    "resolvedResumeScheduledAt": "2026-04-15T10:00:00Z"
  }
}

```

10. Step 6 - BPMN Runs drools-decide

The BPMN should run drools-decide again as a guard because state may have changed between API validation and Camunda execution.

Service task:

```

<bpmn:serviceTask id="Task_drools_validate"
  name="Validate pause policy"
  camunda:type="external"
  camunda:topic="drools-decide" />

```

Input variables:

```

{
  "kjarPackage": "rules.subscription.pause.wik",
  "requestedDecisions": [
    "validate-preconditions",
    "resolve-duration-mode",
    "compute-pause-fee-applicability"
  ],
  "factsJson": "{...fresh subscription state, pause reason, config, open pause check...}"
}

```

If rejected:

```

kafka-emit SubscriptionPauseRejected
subscription-operation-finalize FAILED

```

11. Step 7 - BPMN Moves Subscription To Pending Pause

Service task topic:

```

sub-lm-put-pending-status

```

Worker input:

```

{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "toStatus": "PENDING_PAUSE",
  "currentTransitionType": "PAUSE",
  "currentTransitionReasonCode": "CUSTOMER_VACATION"
}

```

Worker output:

```

{
  "priorStatusSnapshot": {
    "statusCode": "ACTIVE"
  },
  "lmEventId": "lmevt_01HZ_PENDING_PAUSE"
}

```

Compensation later uses the priorStatusSnapshot if the BPMN must revert.

12. Step 8 - BPMN Computes Billing Context

Service task topic:

```

sub-lm-compute-cycle-window

```

Worker input:

```

{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "referenceDate": "2026-03-15T10:00:00Z"
}

```

Worker output:

```

{
  "cycleWindow": {
    "cycleStart": "2026-03-01",
    "cycleEnd": "2026-03-31",
    "cycleAnchorDay": 1
  }
}

```

13. Step 9 - BPMN Sends Billing Intent

Service task topic:

```
bil01-emit-intent
```

Worker input:

```
{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "intentKind": "PAUSE_INTENT",
  "intentContext": {
    "pauseReasonCode": "CUSTOMER_VACATION",
    "durationMode": "FIXED_DAYS",
    "durationDays": 31,
    "resumeScheduledAt": "2026-04-15T10:00:00Z",
    "pausesBillCycle": true,
    "suspendsFulfillment": true,
    "originIntent": "CUSTOMER_REQUESTED_PAUSE"
  },
  "cycleWindow": {
    "cycleStart": "2026-03-01",
    "cycleEnd": "2026-03-31"
  }
}
```

Expected WIK v1.0 output:

```
{
  "intentDecisionId": "bidec_01HZ_KAMAU_PAUSE",
  "lineItems": [],
  "totals": {
    "net": 0,
    "currency": "KES"
  }
}
```

14. Step 10 - BPMN Evaluates Optional Payment Gateway

Gateway condition:

```
${pauseFeeApplicable == true}
```

For WIK v1.0 seeded reasons:

```
pauseFeeApplicable == false
```

So the BPMN skips payment and continues to fulfillment.

If a future reason has a fee, the BPMN runs:

```
bil01-charge-pre-authorize
wait payment-confirmed with boundary timer P7D
bil01-confirm-charge
```

Payment timeout route:

```
bil01-revert-pending-charge
sub-lm-revert-to-prior
kafka-emit SubscriptionPauseRejected
subscription-operation-finalize FAILED
```

These names are worker topics or BPMN message names, not shell commands.

15. Step 11 - BPMN Calls FUL-04 To Suspend Service

Service task topic:

```
ful-call-restrict
```

Worker input:

```
{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "intent": "SUSPEND_SERVICE",
  "reasonCategory": "CUSTOMER_VACATION",
  "suspendsFulfillment": true,
  "fulfillmentCorrelationId": "subop_01HZ_KAMAU_PAUSE:pause"
}
```

The worker calls FUL-04. FUL-04 may return quickly or asynchronously depending on network adapters.

The BPMN then waits for one of:

```
fulfillment-completed  
fulfillment-failed
```

Correlation key:

```
fulfillmentCorrelationId = subop_01HZ_KAMAU_PAUSE:pause
```

16. Step 12 - Fulfillment Message Correlator Wakes BPMN

FUL-04 publishes or calls back with:

```
{  
  "fulfillmentCorrelationId": "subop_01HZ_KAMAU_PAUSE:pause",  
  "terminalState": "FULLY_FULFILLED",  
  "completedAt": "2026-03-15T10:00:05Z"  
}
```

The message correlator calls Camunda:

```
POST /engine-rest/message  
Content-Type: application/json  
  
{  
  "messageName": "fulfillment-completed",  
  "correlationKeys": {  
    "fulfillmentCorrelationId": {  
      "value": "subop_01HZ_KAMAU_PAUSE:pause",  
      "type": "String"  
    }  
  }  
}
```

If FUL-04 fails, correlate fulfillment-failed and route to compensation.

17. Step 13 - BPMN Clears Active Restrictions

Service task topic:

```
sub-lm-put-active-restrictions
```

Worker input:

```
{  
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",  
  "restrictionsOperationKind": "CLEAR_ALL"  
}
```

This removes active restrictions because the subscription is becoming fully suspended, not partially restricted.

18. Step 14 - BPMN Writes Pause History And Audit Fields

Service task topic:

```
sub-lm-put-master-fields
```

Worker input:

```
{  
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",  
  "fieldsJson": {  
    "last_status_changed_at": "2026-03-15T10:00:05Z"  
  },  
  "pauseHistoryRow": {  
    "origin_intent": "CUSTOMER_REQUESTED_PAUSE",  
    "system_managed": false,  
    "pause_reason_code": "CUSTOMER_VACATION",  
    "pause_reason_snapshot": {  
      "code": "CUSTOMER_VACATION",  
      "customer_self_service_eligible": true,  
      "max_duration_days": 90,  
      "open_ended_allowed": false,  
      "pauses_bill_cycle": true,  
      "suspends_fulfillment": true  
    },  
    "duration_mode": "FIXED_DAYS",  
    "duration_days": 31,  
    "resume_scheduled_at": "2026-04-15T10:00:00Z",  
    "pause_fee_billable_event_id": null,  
  }  
}
```

```

        "suspended_at": "2026-03-15T10:00:05Z",
        "actual_resume_at": null,
        "pause_correlation_id": "subop_01HZ_KAMAU_PAUSE",
        "pause_actor_user_id": "cc-agent-wik-007",
        "pause_actor_role": "CALL_CENTER",
        "notes": "Customer travelling to Mombasa for holiday season"
    }
}

```

Implementation guidance:

- If SUB-LM and pause workflow persistence are in one module, this topic can call one application service that writes both audit fields and pause history in one transaction.
- If they are split, this topic should call approved internal APIs or a dedicated orchestration service. Do not hide a cross-service write behind raw SQL.
- Store `pause_reason_snapshot` so later reporting and resume logic do not change meaning if the reason catalog changes.

19. Step 15 - BPMN Commits Subscription State

Service task topic:

```
sub-lm-commit-state
```

Worker input:

```

{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "expectedFromStatus": "PENDING_PAUSE",
  "toStatus": "SUSPENDED"
}

```

Only after this succeeds can the BPMN emit `SubscriptionPaused`.

20. Step 16 - BPMN Emits Event

Service task topic:

```
kafka-emit
```

Topic:

```
sophix.subscription.subscription.paused
```

Payload:

```

{
  "eventType": "SubscriptionPaused",
  "eventVersion": "1.0",
  "operationId": "subop_01HZ_KAMAU_PAUSE",
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "operatorCode": "WIK",
  "customerId": "cust_01HZ_KAMAU",
  "pauseReasonCode": "CUSTOMER_VACATION",
  "durationMode": "FIXED_DAYS",
  "resumeScheduledAt": "2026-04-15T10:00:00Z",
  "suspendedAt": "2026-03-15T10:00:05Z",
  "pausesBillCycle": true,
  "suspendsFulfillment": true
}

```

Write through the outbox. Do not block BPMN completion on Kafka broker availability.

21. Step 17 - BPMN Sends Notification

Service task topic:

```
notification-send
```

Worker input:

```

{
  "recipientKind": "customer",
  "recipientRef": "cust_01HZ_KAMAU",
  "notificationEventKind": "SUBSCRIPTION_PAUSED",
  "eventContext": {
    "pauseReasonCode": "CUSTOMER_VACATION",

```

```

        "durationMode": "FIXED_DAYS",
        "resumeScheduledAt": "2026-04-15T10:00:00Z",
        "originIntent": "CUSTOMER_REQUESTED_PAUSE"
    }
}

```

If notification fails, log it and continue. Do not compensate a successful service pause because of notification failure.

22. Step 18 - BPMN Finalizes Operation

Service task topic:

```
subscription-operation-finalize
```

Worker input:

```

{
  "operationId": "subop_01HZ_KAMAU_PAUSE",
  "finalState": "COMPLETED"
}

```

Final API-visible status:

```

{
  "operationId": "subop_01HZ_KAMAU_PAUSE",
  "finalState": "COMPLETED",
  "subscriptionStatus": "SUSPENDED"
}

```

23. Compensation Flow Details

23.1 Drools Rejection

No mutation should have happened yet.

Flow:

```

kafka-emit SubscriptionPauseRejected
subscription-operation-finalize FAILED

```

23.2 Billing Or Payment Failure

If billing intent fails before a charge exists:

```

sub-lm-revert-to-prior
kafka-emit SubscriptionPauseRejected
subscription-operation-finalize FAILED

```

If a fee charge exists or payment times out:

```

bil01-revert-pending-charge
sub-lm-revert-to-prior
kafka-emit SubscriptionPauseRejected
subscription-operation-finalize FAILED

```

23.3 Fulfillment Failure

If FUL-04 returns fulfillment-failed:

```

bil01-revert-pending-charge if billableEventId exists
sub-lm-revert-to-prior
kafka-emit SubscriptionPauseRejected
subscription-operation-finalize FAILED

```

23.4 Commit Failure After Network Suspension

If pause history write or sub-lm-commit-state repeatedly fails, route to manual recovery.

Reason:

The network may already be suspended. Automatic rollback can be unsafe because the workflow may not know whether downstream suspension fully completed.

24. Manual Smoke Test

1. Seed WIK subscription_operation_config, subscription_pause_config, and pause_reason.

2. Deploy Drools package rules `rules.subscription.pause.wik`.
3. Deploy BPMN sub-pause-wik.
4. Start worker apps subscribed to required topics.
5. Create or identify an ACTIVE WIK subscription.
6. Send the pause API request with `CUSTOMER_VACATION` and `durationDays=31`.
7. Confirm API returns 202 and an operation id.
8. Confirm Camunda process instance key is `sub-pause-wik`.
9. Confirm subscription status becomes `PENDING_PAUSE`.
10. Confirm BIL-01 receives `PAUSE_INTENT` and returns no fee.
11. Correlate fulfillment-completed.
12. Confirm `subscription_pause_history` has an open row with `actual_resume_at = NULL`.
13. Confirm subscription status becomes `SUSPENDED`.
14. Confirm outbox contains `SubscriptionPaused`.
15. Confirm operation final state is `COMPLETED`.

25. Common Mistakes

- Treating worker-topic names as shell commands.
- Removing the optional payment gate because WIK v1.0 has no fee.
- Reading current subscription status from cache.
- Letting customer self-service use admin-only reasons.
- Forgetting the open pause check.
- Not snapshotting the pause reason into history.
- Emitting `SubscriptionPaused` before `sub-lm-commit-state`.
- Failing the whole operation because NOT-01 notification failed.
- Assuming scheduled resume works before RESUME supports `SCHEDULED_AUTO_RESUME`.